Mobile Application Development (MAD)

Mids Notes | Kotlin + Android Studio | Bilkul shuru se, 5th class ke bachay ki tarah | Har topic: Definition, Syntax, Example

Pehle yeh parho: Aap ko is subject ka kuch nahi pata, koi baat nahi. Yeh notes aise likhe hain jaise kisi chhote bachay ko samjha rahe hon. Har topic mein **pehle asaan baat** (kahani ya misaal), phir **neela box** (Definition jo paper mein likhni hai, English mein), phir **Syntax** (likhne ka tareeqa) aur **Example** (chalne wala code). Code English mein hi hota hai, usay copy karna hai, samajhna hai.

Paper ke 7 must-come sawal aur yeh notes mein kahan hain:

Sawal	Kahan hai
1. Kotlin code program	Part 3 (Kotlin basics) aur Part 4 (OOP), aur Part 5 mein 10 tayyar programs.
2. Screen UI + Kotlin + XML code	Part 6 (Views) aur Part 7 (6 complete apps jin ka XML aur Kotlin dono diya hai) aur Part 9 (lifecycle app).
3. Linear, Relative, Constraint layout	Part 8.
4. Activity ke 7 methods	Part 9.
5. Architecture	Part 2.
6. Android Studio intro + WhatsApp/Facebook ki category	Part 1.
7. Intent (sirf types aur methods)	Part 10.

Yaad rakho: Is subject mein tin cheezein sab se zyada marks laati hain: **code likhna**, **UI banana (XML + Kotlin)** aur **lifecycle ke methods**. Inhein ratta nahi, samajh kar yaad karo, kyunke code ek jaisa pattern follow karta hai.

Part 1: Android Studio, Mobile Platforms aur App Categories

1.1 Mobile Platform kya hai?

Socho ek school hai. Us mein building, teachers, kitaabein aur timetable sab hain, tab hi bachay parh sakte hain. Mobile platform bhi aisi hi 'school ki imarat' hai: woh poora maahol jis mein mobile apps **banti bhi hain aur chalti bhi hain**. Is mein operating system, banane ke tools, aur apps bantne ki dukaan sab shamil hain.

Definition (paper mein likho):
A mobile platform is a software environment that provides the operating system, tools, frameworks and services required to develop and run mobile applications. Examples: Android, iOS, HarmonyOS, KaiOS.

Mobile platform ke 5 hisse

Hissa	Aasan matlab	Example
Operating System (OS)	Mobile ka manager jo hardware aur software ko chalata hai.	Android, iOS
Development Tools	Developer app banane ke liye jo software use karta hai.	Android Studio, Xcode
SDK	Software Development Kit: tool box jis mein libraries, APIs aur debugging tools hote hain.	Android SDK
Application Framework	Pehle se bane hue tayyar parts jo app mein seedha use hote hain.	Button, TextView, Activity
App Store	Jahan se log app download karte hain.	Google Play Store, Apple App Store

1.2 Android vs iOS

Android Google ka mobile operating system hai (Linux kernel par bana). Isme apps **Kotlin ya Java** mein bante hain aur **Android Studio** mein likhte hain. Bohat si companies ke phones mein chalta hai (Samsung, Xiaomi, Oppo, Vivo). **iOS** Apple ka operating system hai, sirf iPhone aur iPad mein. Isme apps **Swift ya Objective-C** mein bante hain aur **Xcode** mein likhte hain.

Pehlu	Android	iOS
Developer	Google / Open Handset Alliance	Apple
Devices	Bohat si companies ke	Sirf Apple ke
Language	Kotlin (aur Java)	Swift (aur Objective-C)
IDE	Android Studio	Xcode
App store	Google Play	App Store
Customization	Zyada (open)	Kam (controlled)
Hardware variety	Bohat zyada	Kam

Yaad rakho: Android = open aur diverse ecosystem. iOS = controlled aur tightly integrated ecosystem. (Yeh line slides mein 'Important Exam Point' hai.)

1.3 App banane ke 4 tareeqe (Development Approaches)

Ek app banane ke chaar tareeqe hote hain. Isay is tarah samjho: aap ko ek pizza banana hai. Ya to **har ghar ke liye alag recipe** (native), ya **ek recipe sab ghar ke liye** (cross-platform), ya **website ko app ke dibbe mein daal do** (hybrid), ya **website ko hi app jaisa bana do** (PWA).

Tareeqa	Aasan matlab	Fayde / Nuqsan	Example
Native	Ek khaas operating system ke liye khaas tools se bani. Android: Kotlin + Android Studio. iOS: Swift + Xcode.	Tez, device ki saari features, behtareen experience. Nuqsan: Android aur iOS ke liye alag alag banani parti hai.	WhatsApp, Facebook (mainly native).
Cross-platform	Ek hi code se kayi platforms (Android + iOS). Tools: Flutter, React Native, .NET MAUI.	Kam waqt, kam cost, code reuse. Nuqsan: kabhi extra kaam, performance kabhi thori kam.	Flutter se bani app.

Tareeqa	Aasan matlab	Fayde / Nuqsan	Example
Hybrid	Web technologies (HTML, CSS, JavaScript) ko app ke andar chalana. Tools: Ionic, Capacitor.	Web developers ke liye aasan. WebView isi se juri hai.	Ionic app.
PWA (Progressive Web App)	Website jo app jaisi lagti hai: offline chalti hai, install hoti hai, notifications deti hai.	Install ke baghair browser se chal jati hai.	University portal jo phone mein install ho jaye.

1.4 Android Studio kya hai? (Introduction)

Agar aap ko cake banani hai to aap ko **kitchen** chahiye jis mein oven, bartan aur ingredients hon. Isi tarah Android app banane ke liye **Android Studio** woh kitchen hai. Yahan app likhte hain, chala kar dekhte hain, bugs dhoondte hain, aur phir phone par bhejte hain.

Definition (paper mein likho):
Android Studio is the official Integrated Development Environment (IDE) for Android application development. It is used to write, build, debug and test Android applications.

Cheez	Matlab	Aasan misaal
Kotlin	Programming language. Hum is mein app ka logic (kaam) likhte hain.	Cook ki recipe
Android Studio	IDE (kaam karne ki jagah). Code likhna, run karna, debug karna.	Kitchen
Android SDK	Tools aur APIs ka box jo app banane ke liye zaroori hain.	Kitchen ke bartan aur tools
Emulator	Computer ke andar chalne wala nakli phone, taake asal phone ke baghair test ho.	Practice ke liye nakli stove
Gradle	Woh system jo code ko jod kar asal app (APK) bana deta hai.	Oven jo recipe ko cake bana de
Jetpack libraries	Tayyar libraries jo modern apps aasani se banane mein madad deti hain (lifecycle, navigation, Room, Compose).	Ready-made masala

Android Studio mein project ke andar kya hota hai?

Jab aap naya project banate ho (New Project, Empty Views Activity, language Kotlin), to yeh folders milte hain:

File / Folder	Kaam
MainActivity.kt	java folder ke andar. Isme Kotlin code (app ka logic) likhte hain.
activity_main.xml	res/layout folder mein. Isme screen ka design (UI) likhte hain.
AndroidManifest.xml	App ka 'shanakhti card': app ka naam, kaun si activities hain, permissions.
res/drawable	Images aur icons.
res/values/strings.xml	Text ke strings (jaise app ka naam).
build.gradle	App ko banane ki settings aur libraries.

Workflow: Android Studio -> Android SDK -> Jetpack Libraries -> Application UI -> Kotlin Source Code -> Android Application -> Android Device / Emulator.

Yaad rakho: Slide ki ek **correction**: Android Studio, SDK, Jetpack aur UI development ke hisse hain, Android OS ki layers nahi. Asal OS architecture alag hai (Part 2 mein).

1.5 Sawal: WhatsApp aur Facebook jaisi apps Android ya iOS ki kaunsi category mein aati hain?

Yeh sawal teen tareeqon se poocha ja sakta hai, is liye teeno jawab yaad rakho:

Nazar	Jawab	Detail
1) Platform ke hisab se	WhatsApp aur Facebook **dono Android aur iOS** par chalti hain. Yeh kisi ek platform ki nahi, **dono platforms ki apps** hain.	Android: Google Play. iOS: App Store.
2) Development approach ke hisab se	Inhein **native mobile applications** kaha jata hai (Android ke liye Kotlin/Java, iOS ke liye Swift/Objective-C). Facebook ke kuch hisson mein React Native bhi istemal hota hai, magar bunyadi tor par native apps hain.	Native: tez, camera, GPS, notifications sab features.
3) Purpose (kaam) ke hisab se	WhatsApp = **Communication / Messaging app**. Facebook = **Social Networking app**. Daraz = E-commerce. Easypaisa = Mobile banking / payment.	App ki 'category' Play Store par isi tarah likhi hoti hai.

Paper mein aise likho (model answer):

WhatsApp and Facebook are mobile applications available on both Android and iOS. They are mainly native applications, built with the native tools of each platform (Kotlin/Java for Android and Swift/Objective-C for iOS). By purpose, WhatsApp is a communication (messaging) application and Facebook is a social networking application. They use cloud services, notifications, authentication, camera and GPS features.

Practical activity (slide se): kisi bhi app ke liye yeh 10 sawal solve karo: kaunsa platform? Android/iOS/dono? app ki qisam? cloud use karti hai? AI? GPS? notifications? authentication? biometric/device features? kaunsa development approach munasib hai?

App	Platform	Qisam	Cloud	Khaas features	Approach
WhatsApp	Android + iOS	Communication	Haan (messages cloud se)	Haan (notifications, login OTP)	Native
Instagram	Android + iOS	Social media	Haan	AI recommendations, camera	Native
Daraz	Android + iOS	E-commerce	Haan	Notifications, login, payment	Native ya cross-platform
Google Maps	Android + iOS	Maps / Navigation	Haan	GPS lazmi	Native
Easypaisa	Android + iOS	Mobile banking	Haan	Authentication, biometric, security	Native

Part 2: Android Architecture (layers wala sawal)

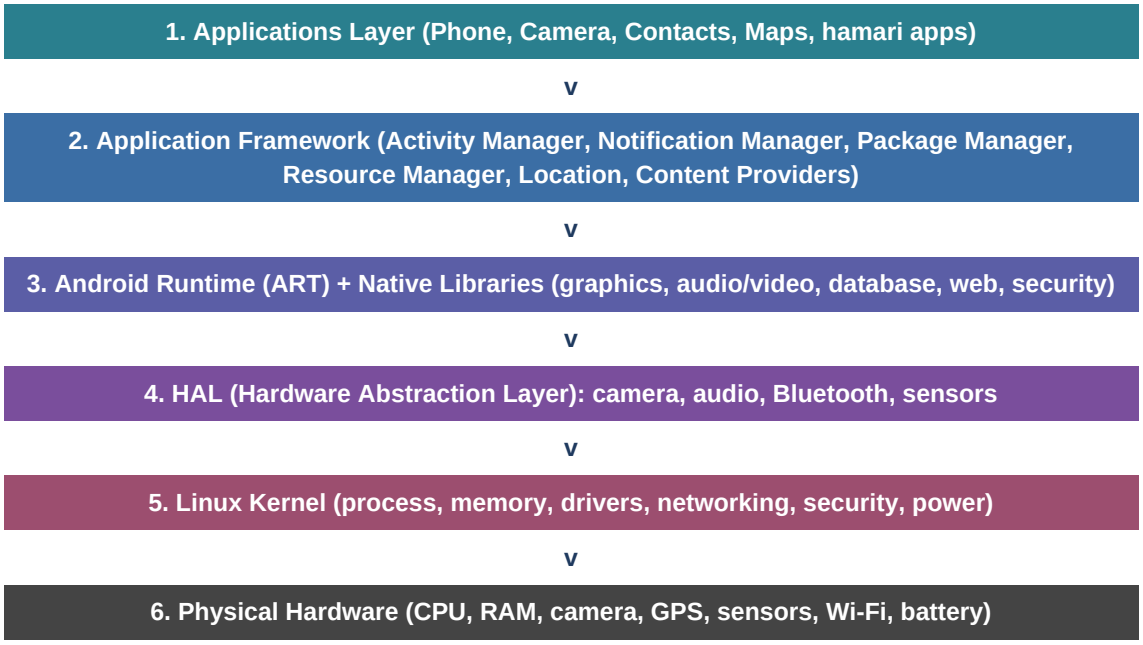
2.1 Android Architecture kya hai?

Ek **imarat** ke baare mein socho: sab se neeche bunyaad (foundation), us ke upar manzilein, aur sab se upar log rehte hain. Android bhi aisi hi ek imarat hai jis mein **7 manzilein (layers)** hain. Har layer apne neeche wali layer se madad leti hai. Sab se upar wo apps hain jo hum istemal karte hain, aur sab se neeche mobile ka asal hardware.

Definition (paper mein likho):
Android Architecture is a layered software structure that provides the runtime, libraries, system services, hardware interfaces and kernel functionality required to develop and run Android applications.

Architecture ka diagram (paper mein yehi banana hai)

Upar se neeche tarteeb yaad karo: **Applications -> Application Framework -> ART + Native Libraries -> HAL -> Linux Kernel -> Hardware.**

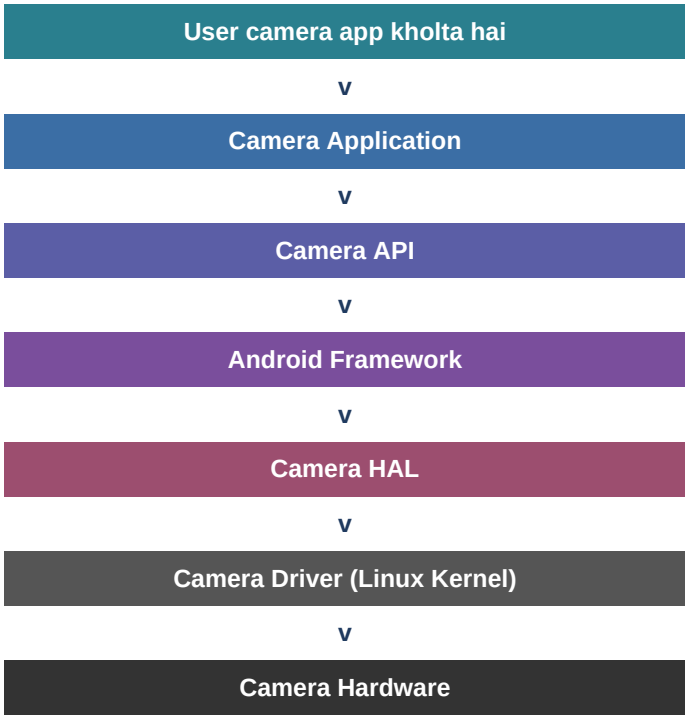


Har layer ko aasan zubaan mein samjho

Layer	Aasan matlab	Example
1. Applications	Sab se upar wali layer. Jo apps user dekhta aur chalata hai.	Phone, Messages, Camera, Maps, Games, banking apps, aur **hamari Kotlin app** (Student Attendance app).
2. Application Framework	Android ki taraf se diye gaye tayyar **services aur APIs**. Developer ko hardware se seedha baat nahi karni parti. Services: **Activity Manager** (screens aur unka lifecycle), **Notification Manager**, **Package Manager** (installed apps), **Resource Manager** (strings, images), **Location Services**, **Content Providers** (data sharing).	App ko location chahiye to developer Android ki location API use karta hai, GPS hardware se seedha baat nahi karta.
3a. ART (Android Runtime)	Woh jagah jahan **app ka code chalta hai**. Isay runtime kehte hain.	Kotlin code build ho kar app banta hai, phir ART us ko chalata hai.
3b. Native Libraries	Neeche ke level ki libraries jo graphics, audio/video, database, web, security ke kaam karti hain.	Camera app in libraries se tasveer process karti hai.
4. HAL	Hardware Abstraction Layer: ek **standard darwaza** jo Android ke upar wale hisson ko alag alag companies ke hardware se baat karne deta hai. Camera, audio, Bluetooth, sensors ke liye.	Samsung aur Xiaomi ke camera alag, magar Android ke liye interface ek jaisa.
5. Linux Kernel	Android ki **bunyaad**. Core kaam: process management, memory management, device drivers, networking, security, power management.	Jab app hardware se kaam leti hai to aakhir mein kernel hi usay sambhalta hai.
6. Hardware	Asal physical cheezein.	CPU, RAM, storage, camera, microphone, GPS, sensors, Wi-Fi, Bluetooth, display, battery.

Yaad rakho: Kotlin architecture ki layer nahi hai. Kotlin sirf wo programming language hai jis se hum app banate hain.

Camera app ka example (flow)



2.2 Android ke 4 Application Components

Layers to OS ki baat thi. Ab app ke apne **4 buniyaadi hisse** samjho. Ek restaurant ki tarah sochiye:

Component	Aasan matlab	Example
Activity	Ek **screen** jo user ko nazar aati hai aur jis se wo interact karta hai.	Waiter jo customer se baat karta hai. Login screen, Home screen, Profile screen.
Service	Aisa kaam jo **bina screen ke** background mein chalta rahe.	Kitchen mein chalta kaam. Music app band screen par bhi gaana chalati hai.
Broadcast Receiver	App ko **announcements/events** sunne deta hai.	Restaurant ki ghanti. Battery low, internet connect, phone restart.
Content Provider	Data ko doosri apps ke saath **control ke saath share** karna.	Store room ki chaabi jo sirf ijazat wale logon ko milti hai. Contacts ka data.

Definition (paper mein likho):
Android applications are built around four fundamental components: Activity (user screen), Service (background task), Broadcast Receiver (responds to system events) and Content Provider (shares structured data between apps).

Key definitions jo yaad karni hain

Term	Definition
Kotlin	Programming language
Android Studio	Development environment (IDE)
Android SDK	Development tools and APIs
Activity	User-facing application component (screen)
Intent	Messaging mechanism to request an action
ART	Android Runtime
HAL	Interface between Android software and hardware-specific implementations
Linux Kernel	Foundation providing core operating-system functionality

Part 3: Kotlin Programming (shuru se)

Kotlin kya hai? Jaise insaan aapas mein Urdu ya English mein baat karte hain, waise hum computer se **programming language** mein baat karte hain. Kotlin ek aisi language hai jis mein hum Android app ke liye hukm (code) likhte hain. Yeh chhoti, saaf aur mehfooz hai.

3.1 Kotlin kya hai aur Android ke liye kyun?

Definition (paper mein likho):
Kotlin is a modern, statically typed programming language developed by JetBrains. It is officially supported by Google for Android development and is fully interoperable with Java.

Yaad rakho: Kotlin kyun: kam code, null-safety, aasan syntax, modern features, Java ke saath chalti hai, Android Studio mein zabardast support.

3.2 Pehla program aur main function

Har program ek jagah se shuru hota hai, jaise school ki ghanti se din shuru hota hai. Kotlin mein yeh jagah **fun main()** hai. **println()** screen par kuch likh kar dikhata hai.

Definition (paper mein likho):
The main function is the starting point of a Kotlin program. println() displays output on the screen and moves to a new line.

Syntax:

```
fun main() {
    println("Hello, Kotlin!")
}
```

Example:

```
fun main() {
    println("Welcome to Mobile Application Development")
}
```

Output: Welcome to Mobile Application Development

Yaad rakho: Kotlin mein semicolon (;) lagana zaroori nahi hota. Java mein hota hai: int age = 20; Kotlin mein: val age = 20

3.3 print() aur println(), Comments

println likhne ke baad **nayi line** par chala jata hai. **print** usi line mein rehta hai. **Comment** woh note hai jo programmer apne liye likhta hai, computer usay chhor deta hai, jaise copy ki margin mein pencil se likhi baat.

Definition (paper mein likho):
print() displays output without a new line; println() displays output and adds a new line. Comments are text ignored by the compiler: single-line starts with //, multi-line starts with /* and ends with */.

Syntax:

```
print("text")
println("text")
// single line comment
/* multi
   line comment */
```

Example:

```
fun main() {
    println("Hello World!")
    println("I am learning Kotlin.")
    print("No new line ")
    print("here")
    // This is a comment
}
```

Output: Hello World!
I am learning Kotlin.
No new line here

3.4 Variables: val aur var

Variable ek **dibba** hai jis par naam likha hota hai aur andar kuch rakha hota hai. **val** ek aisa dibba hai jis par **taala laga hai**: andar ki cheez badal nahi sakte. **var** ka dibba **khula** hai: cheez badal sakte hain. Jaise aap ka naam (val, badalta nahi) aur aap ki umar (var, har saal badalti hai).

Definition (paper mein likho):
A variable is a named memory location used to store data. val is a read-only reference (cannot be reassigned); var is a mutable variable (can be changed).

Syntax:

```
val variableName = value
var variableName = value
```

Example:

```
val name = "Ali"      // cannot change
var age = 20         // can change
age = 21             // OK
// name = "Ahmed"    // ERROR: val cannot be reassigned
```

Yaad rakho: Asaan usool: pehle hamesha val use karo. Jab value badalni ho tab var.

Keyword	Matlab
val	Read-only. Dobara value nahi de sakte.
var	Mutable. Value badal sakte hain.

3.5 Data Types

Data type batata hai dibbe mein **kis qisam ki cheez** hai: poora number, decimal number, haan/na, ek harf ya poora lafz.

Definition (paper mein likho):
A data type specifies what kind of data a variable can store.

Syntax:

```
val age: Int = 20
val price: Double = 250.50
val isStudent: Boolean = true
val grade: Char = 'A'
val name: String = "Ali"
```

Type	Matlab	Example
Int	Poora number	25
Long	Bohat bara poora number	100000L
Float	Chhota decimal	3.14f
Double	Bara decimal	3.14159
Boolean	Sach ya jhoot	true / false
Char	Aik harf (single quote)	'A'
String	Lafz ya jumla (double quote)	"Kotlin"

3.6 Type Inference

Kabhi kabhi hum likhte nahi ke dibba kis qisam ka hai, Kotlin khud andaza laga leta hai, jaise aap dekh kar samajh jate ho ke yeh seb hai ya kela.

Definition (paper mein likho):
Type inference is Kotlin's ability to automatically detect the data type of a variable from its value.

Syntax:

```
val variable = value
```

Example:

```
val age = 20        // Int
val name = "Ali"    // String
val price = 250.5   // Double
```

3.7 String Templates

Text ke andar variable ki value likhni ho to **$** ka nishan lagate hain. Agar hisaab karna ho to **${ }** ke andar likhte hain.

Definition (paper mein likho):
String templates allow variables and expressions to be inserted into a string using $ or ${ }.

Syntax:

```
"text $variable"
"text ${expression}"
```

Example:

```
val name = "Ali"
val age = 20
println("My name is $name")
println("I am $age years old")
val a = 10
val b = 20
println("Sum = ${a + b}")
```

Output: My name is Ali
I am 20 years old
Sum = 30

3.8 Operators (+ - * / %)

Operators wo nishan hain jo hisaab karte hain, bilkul math ki tarah. Khaas: **%** baqi (remainder) batata hai: 10 % 3 = 1 kyunke 10 ko 3 se taqseem karein to 1 bachta hai.

Definition (paper mein likho):
Arithmetic operators perform mathematical calculations: + (addition), - (subtraction), * (multiplication), / (division), % (remainder).

Syntax:

```
a + b   a - b   a * b   a / b   a % b
```

Example:

```
val a = 10
val b = 3
println(a + b)   // 13
println(a - b)   // 7
println(a * b)   // 30
println(a / b)   // 3
println(a % b)   // 1
```

3.9 Comparison aur Logical Operators

Comparison: do cheezon ka muqabla, jawab hamesha true ya false. **Logical:** kayi shartein jodna. **&&** matlab 'aur' (dono sach ho), **||** matlab 'ya' (koi ek sach ho), **!** matlab 'ulta'.

Definition (paper mein likho):
Comparison operators (greater than, less than, greater or equal, less or equal, ==, !=) compare two values and return true or false. Logical operators (&&, ||, !) combine conditions: AND, OR, NOT.

Syntax:

```
a > b   a < b   a == b   a != b
cond1 && cond2    cond1 || cond2    !cond
```

Example:

```
val a = 10
val b = 20
println(a > b)    // false
println(a < b)    // true
println(a == b)   // false
println(a != b)   // true
val age = 22
val student = true
println(age >= 18 && student)   // true
```

Output: false, true, false, true, true

3.10 Input lena: readLine()

Kabhi program user se kuch poochta hai, jaise teacher aap se naam poochte hain. Wo **readLine()** se hota hai. User jo likhta hai wo text (String) ban kar aata hai. Agar number chahiye to **.toInt()** lagate hain.

Definition (paper mein likho):
readLine() reads a line of text entered by the user. toInt() converts the text into an integer.

Syntax:

```
val variable = readLine()
val number = readLine()!!.toInt()
```

Example:

```
fun main() {
    print("Enter your name: ")
    val name = readLine()
    println("Hello $name")
}
```

Output: Agar user Ali likhe to: Hello Ali

3.11 if, else if, else (faisla karna)

Hum roz faislay karte hain: 'agar baarish ho to chhatri lo, warna nahi'. Programming mein yehi **if** hai. 'warna' ko **else** kehte hain, aur 'warna agar' ko **else if**.

Definition (paper mein likho):
Conditional statements allow a program to make decisions. if runs a block when a condition is true, else runs when it is false, and else if tests a new condition.

Syntax:

```
if (condition) {
    // code
} else if (condition2) {
    // code
} else {
    // code
}
```

Example:

```
fun main() {
    val marks = 75
    if (marks >= 80) {
        println("A")
    } else if (marks >= 70) {
        println("B")
    } else if (marks >= 60) {
        println("C")
    } else {
        println("Fail")
    }
}
```

Output: B

3.12 when expression

Agar bohat saare 'agar' hon to if ka dher ban jata hai. **when** ek saaf tareeqa hai, jaise menu: 1 dabao to yeh, 2 dabao to woh. Android mein button, menu aur user ki choice ke liye bohat use hota hai.

Definition (paper mein likho):
when is an expression used to select one of many alternatives. It is a cleaner replacement for multiple if-else statements.

Syntax:

```
when (value) {
    1 -> "one"
    2 -> "two"
    else -> "other"
}
```

Example:

```
fun main() {
    val day = 4
    val result = when (day) {
        1 -> "Monday"
        2 -> "Tuesday"
        3 -> "Wednesday"
        4 -> "Thursday"
        5 -> "Friday"
        6 -> "Saturday"
        7 -> "Sunday"
        else -> "Invalid day."
    }
    println(result)
}
```

Output: Thursday

3.13 Loops: for, while, do-while

Agar aap ko 1 se 100 tak likhna ho to 100 baar likhna bewaqoofi hai. **Loop** ek hi kaam ko baar baar karta hai. **for** tab jab pata ho kitni baar. **while** tab jab shart tak karna ho. **do-while** pehle ek baar karta hai, phir shart dekhta hai.

Definition (paper mein likho):
Loops repeat a block of code. for loops over a range or collection, while repeats while a condition is true, and do-while executes the block at least once before checking the condition.

Syntax:

```
for (i in 1..5) { }
while (condition) { }
do { } while (condition)
```

Example:

```
// for loop
for (i in 1..5) {
    println(i)
}
// for with array
val cars = arrayOf("Volvo", "BMW", "Ford")
for (x in cars) {
    println(x)
}
// while loop
var i = 1
while (i <= 5) {
    println(i)
    i++
}
// do-while loop
var j = 1
do {
    println(j)
    j++
} while (j <= 5)
```

Output: 1 2 3 4 5 (teeno loops mein)

3.14 Functions (kaam ka tukra)

Function ek **chhota mashin** hai jo ek kaam karta hai. Aap bulate ho (call karte ho) to kaam karta hai. Mashin mein cheez daalna = **parameter**. Mashin se nateeja wapas lena = **return value**. Jaise juicer: phal daalo (parameter), juice nikalta hai (return).

Definition (paper mein likho):
A function is a reusable block of code that performs a specific task and runs only when it is called. Functions can take parameters and return a value.

Syntax:

```
fun functionName(param: Type): ReturnType {
    return value
}
```

Example:

```
// no parameter
fun greet() {
    println("Welcome to Kotlin")
}
// with parameter
fun greetUser(name: String) {
    println("Hello $name")
}
// with return value
fun add(a: Int, b: Int): Int {
    return a + b
}
fun main() {
    greet()
    greetUser("Ali")
    val result = add(10, 20)
    println(result)
}
```

Output: Welcome to Kotlin, Hello Ali, 30

3.15 Null Safety

Null ka matlab 'kuch nahi' (khali dibba). Khali dibba kholne ki koshish karo to app crash ho jati hai. Kotlin **pehle se bacha leta hai**: normal variable null nahi ho sakta. Agar null rakhna ho to type ke baad **?** lagate hain, aur use karte waqt **?.** (safe call).

Definition (paper mein likho):
Null safety is a Kotlin feature that prevents NullPointerException. A variable can hold null only if its type is marked with ?. The safe call operator ?. accesses members safely.

Syntax:

```
var name: String? = null
name?.length
```

Example:

```
var name: String = "Ali"        // cannot be null
var city: String? = null        // can be null
println(city?.length)           // prints null, no crash
```

3.16 Arrays, List, Set, Map

Agar 30 students ke naam rakhne hon to 30 variable banana mushkil hai. **Array/List** ek **lambi almari** hai jis ke khano mein cheezein rakhte hain, aur har khane ka number hota hai jo **0 se shuru** hota hai. **Set** mein dohraye hue (duplicate) naam nahi aate. **Map** ek dictionary hai: ek 'key' (lafz) ka ek 'value' (matlab).

Definition (paper mein likho):
An array stores multiple values of the same type. A List stores ordered items (listOf is read-only, mutableListOf can be changed). A Set stores unique values. A Map stores key-value pairs.

Syntax:

```
arrayOf(1, 2, 3)
listOf("a", "b")
mutableListOf("a")
setOf(1, 2)
mapOf("key" to "value")
```

Example:

```
val numbers = arrayOf(10, 20, 30, 40)
println(numbers[0])                // 10
val students = mutableListOf("Ali", "Ahmed")
students.add("Sara")
println(students)                  // [Ali, Ahmed, Sara]
val set = setOf(10, 20, 20, 30)
println(set)                       // [10, 20, 30]
val s = mapOf("Name" to "Ali", "RollNo" to "101")
println(s["Name"])                 // Ali
```

3.17 Exception Handling (try, catch, finally)

Exception matlab program chalte hue achanak masla, jaise 10 ko 0 se taqseem karna. **try** mein khatarnak kaam rakhte hain, masla aaye to **catch** sambhal leta hai taake app crash na ho. **finally** hamesha chalta hai.

Definition (paper mein likho):
Exception handling manages unexpected runtime errors using try, catch and finally so that the program does not crash.

Syntax:

```
try {
    // risky code
} catch (e: Exception) {
    // handle error
} finally {
    // always runs
}
```

Example:

```
fun main() {
    try {
        val result = 10 / 0
        println(result)
    } catch (e: Exception) {
        println("An error occurred")
    }
}
```

Output: An error occurred

Part 4: Kotlin OOP (Classes aur Objects)

4.1 OOP kya hai?

Pehle program 'kaam ki list' hote the. OOP mein hum cheezon (objects) ke baare mein sochte hain, jaise asal duniya mein: ek gari ke paas **brand, rang** (properties) hote hain aur woh **chal sakti hai, ruk sakti hai** (functions). OOP se code saaf, dubara istemal hone wala aur sambhalna aasan hota hai (DRY: Don't Repeat Yourself).

Definition (paper mein likho):
Object-Oriented Programming (OOP) is a programming style based on creating objects that contain both data (properties) and behavior (functions).

4.2 Class aur Object

Class ek naksha hai (cookie cutter). **Object** us naksha se bani asal cheez (cookie). Ek Fruit class ho to Apple, Banana, Mango uske objects hain.

Definition (paper mein likho):
A class is a blueprint or template used to create objects. An object is an instance of a class.

Syntax:

```
class ClassName {
    var property = value
}
val obj = ClassName()
```

Example:

```
class Student {
    var name = "Ali"
    var age = 20
}
fun main() {
    val student = Student()   // object
    println(student.name)
    println(student.age)
}
```

Output: Ali
20

4.3 Properties aur Methods

Property object ki maloomat (variable jo class ka hissa ho). **Method (member function)** object ka kaam (function jo class ke andar ho).

Definition (paper mein likho):
A property is a variable that belongs to a class. A method (member function) is a function defined inside a class.

Syntax:

```
class ClassName {
    var property = value
    fun methodName() { }
}
```

Example:

```
class Student {
    var name = "Ali"
    fun displayStudent() {
        println("Student Name: $name")
    }
}
fun main() {
    val student = Student()
    student.displayStudent()
}
```

Output: Student Name: Ali

4.4 Constructor

Jab naya bacha paida hota hai to uska naam usi waqt rakh dete hain. Constructor object banate hi **uski values bhar deta hai**. Kotlin mein class ke naam ke baad bracket () mein properties likh dete hain.

Definition (paper mein likho):
A constructor is a special function used to initialize an object when it is created. The primary constructor is written in parentheses after the class name.

Syntax:

```
class ClassName(val a: Type, val b: Type)
```

Example:

```
class Student(
    val name: String,
    val rollNo: Int,
    val cgpa: Double
) {
    fun display() {
        println("Name: $name")
        println("Roll No: $rollNo")
        println("CGPA: $cgpa")
    }
}
fun main() {
    val student = Student("Ahmed", 101, 3.5)
    student.display()
}
```

Output: Name: Ahmed, Roll No: 101, CGPA: 3.5

4.5 Inheritance (virasat)

Bacha walidain se cheezein virasat mein leta hai. Child class parent class ka data aur kaam **bina dobara likhe** le leti hai. Kotlin mein parent class ko **open** likhna parta hai (warna koi virasat nahi le sakta), aur child mein **: ParentClass()** likhte hain.

Definition (paper mein likho):
Inheritance allows a child (subclass) to acquire the properties and functions of a parent (superclass). The parent class must be declared open.

Syntax:

```
open class Parent { }
class Child : Parent() { }
```

Example:

```
open class Animal {
    fun eat() {
        println("Animal is eating")
    }
}
class Dog : Animal() {
    fun bark() {
        println("Dog is barking")
    }
}
fun main() {
    val dog = Dog()
    dog.eat()
    dog.bark()
}
```

Output: Animal is eating
Dog is barking

4.6 Method Overriding

Child class parent ke kaam ko **apne tareeqe se badal** sakti hai. Parent ke method par **open**, aur child ke method par **override** likhte hain.

Definition (paper mein likho):
Method overriding is when a child class provides its own implementation of a method already defined in its parent class, using the override keyword.

Syntax:

```
open class Parent { open fun f() { } }
class Child : Parent() { override fun f() { } }
```

Example:

```
open class Animal {
    open fun sound() {
        println("Animal makes sound")
    }
}
class Dog : Animal() {
    override fun sound() {
        println("Dog barks")
    }
}
fun main() {
    Dog().sound()
}
```

Output: Dog barks

4.7 Interface

Interface ek **wada (contract)** hai: 'jo class mujhe use karegi, usay yeh kaam karne hi honge'. Isme sirf naam hota hai, kaam likhna class ka zimma.

Definition (paper mein likho):
An interface defines functions or properties that a class must implement.

Syntax:

```
interface Name {
    fun methodName()
}
class MyClass : Name {
    override fun methodName() { }
}
```

Example:

```
interface Vehicle {
    fun start()
}
class Car : Vehicle {
    override fun start() {
        println("Car started")
    }
}
fun main() {
    Car().start()
}
```

Output: Car started

4.8 Data Class

Agar class sirf data rakhne ke liye ho (jaise Student ki maloomat) to **data class** likhte hain. Chhota aur aasan. Android mein UI par dikhane wale data ke liye bohat use hoti hai.

Definition (paper mein likho):
A data class is a class designed mainly to hold data. Kotlin automatically provides useful functions such as toString() and copy().

Syntax:

```
data class Name(val a: Type, val b: Type)
```

Example:

```
data class Student(
    val name: String,
    val rollNo: Int,
    val department: String
)
fun main() {
    val student = Student("Ali", 101, "Computer Science")
    println(student.name)
    println(student.rollNo)
    println(student.department)
}
```

Output: Ali, 101, Computer Science

Part 5: Paper mein aane wale Kotlin Programs (10 tayyar programs)

Yeh woh programs hain jo bar bar poochay jate hain. Har ek ko pehle khud likhne ki koshish karo, phir dekho. Sab ek jaisa pattern follow karte hain: variables -> hisaab/shart -> println.

Program 1: Hello message

```
fun main() {
    println("Welcome to Mobile Application Development")
}
```

Output: Welcome to Mobile Application Development

Program 2: Student information

```
fun main() {
    val name = "Ali"
    val rollNo = 101
    val semester = 5
    val cgpa = 3.5
    println("Name: $name")
    println("Roll No: $rollNo")
    println("Semester: $semester")
    println("CGPA: $cgpa")
}
```

Output: Name: Ali, Roll No: 101, Semester: 5, CGPA: 3.5

Program 3: Total aur average marks

```
fun main() {
    val english = 80
    val computer = 90
    val mathematics = 70
    val total = english + computer + mathematics
    val average = total / 3.0
    println("Total Marks = $total")
    println("Average Marks = $average")
}
```

Output: Total Marks = 240, Average Marks = 80.0

Program 4: Grade calculator (user se marks lo)

```
fun main() {
    print("Enter marks: ")
    val marks = readLine()!!.toInt()
    if (marks >= 80) {
        println("Grade A")
    } else if (marks >= 70) {
        println("Grade B")
    } else if (marks >= 60) {
        println("Grade C")
    } else if (marks >= 50) {
        println("Grade D")
    } else {
        println("Fail")
    }
}
```

Output: Input 78 -> Grade B

Program 5: Even ya Odd

```
fun main() {
    print("Enter a number: ")
    val n = readLine()!!.toInt()
    if (n % 2 == 0) {
        println("$n is Even")
    } else {
        println("$n is Odd")
    }
}
```

Output: Input 7 -> 7 is Odd. (Trick: 2 se taqseem karke baqi 0 bache to even.)

Program 6: Calculator (+ - * /)

```
fun main() {
    val a = 10
    val b = 5
    println("Addition = ${a + b}")
    println("Subtraction = ${a - b}")
    println("Multiplication = ${a * b}")
    println("Division = ${a / b}")
}
```

Output: 15, 5, 50, 2

Program 7: Table (loop se)

```
fun main() {
    val n = 5
    for (i in 1..10) {
        println("$n x $i = ${n * i}")
    }
}
```

Output: 5 x 1 = 5 ... 5 x 10 = 50

Program 8: Factorial (while loop)

```
fun main() {
    val n = 5
    var fact = 1
    var i = 1
    while (i <= n) {
        fact = fact * i
        i++
    }
    println("Factorial of $n = $fact")
}
```

Output: Factorial of 5 = 120

Program 9: Teen numbaron mein se sab se bara

```
fun main() {
    val a = 15
    val b = 40
    val c = 25
    val largest = if (a >= b && a >= c) a
                  else if (b >= c) b
                  else c
    println("Largest = $largest")
}
```

Output: Largest = 40

Program 10: Student class ki list (3 students)

```
class Student(val name: String, val rollNo: Int, val cgpa: Double) {
    fun display() {
        println("$name | $rollNo | $cgpa")
    }
}
fun main() {
    val students = listOf(
        Student("Ali", 101, 3.5),
        Student("Sara", 102, 3.8),
        Student("Ahmed", 103, 3.2)
    )
    for (s in students) {
        s.display()
    }
}
```

Output: Ali | 101 | 3.5, Sara | 102 | 3.8, Ahmed | 103 | 3.2

Part 6: Screen banana: XML + Kotlin (Views)

6.1 XML aur Kotlin ka rishta (sab se pehle yeh samjho)

Ek dukan socho. **XML** dukan ka **sajawat aur naksha** hai: shelves kahan, board kahan, dibbe kahan (screen kaisi nazar aaye). **Kotlin** dukandar ka **kaam** hai: gahak aaya to kya karna hai, paise kaise lene hain (button dabane par kya ho). Dono mil kar dukan chalti hai.

File	Kaam	Example
activity_main.xml	UI design (screen ki shakal)	TextView, EditText, Button, ImageView
MainActivity.kt	Behavior (screen ka kaam)	Button click, input read karna, result dikhana

Definition (paper mein likho):
XML (eXtensible Markup Language) is used in traditional Android Views to define the user interface layout. Kotlin code controls the behavior of the UI elements.

Flow: activity_main.xml (UI design) -> MainActivity.kt (application logic).

6.2 UI aur UX mein farq

	Matlab	Aasan
UI (User Interface)	Jo user **dekhta** hai: rang, buttons, fonts, icons.	Look (shakal)
UX (User Experience)	User ko app **kaisi mehsoos** hoti hai: aasani, navigation, khushi.	Feel (ehsaas)

Real-life example: Do restaurants hain. Dono ka menu card khoobsurat hai (UI achha). Ek mein order jaldi aur aasani se mil jata hai, doosre mein 1 ghanta intezar (UX kharab). UI = menu ki shakal. UX = poora tajurba.

5 UI/UX principles: (1) Simplicity: faltu cheezein na rakho. (2) Consistency: rang, font, buttons ek jaise. (3) Readability: text saaf aur parhne ke qabil. (4) Clear Navigation: user ko pata ho kahan jana hai. (5) Feedback: action ke baad batao kya hua, jaise 'Registration Successful'.

6.3 View aur ViewGroup

View ek chhoti cheez hai jo screen par nazar aati hai (button, text). **ViewGroup** ek **dibba** hai jis mein kayi views rakhte hain (LinearLayout, ConstraintLayout). Jaise almari (ViewGroup) mein kitabein (Views).

Definition (paper mein likho):
A View is a UI element that displays information or allows user interaction (TextView, Button, EditText, ImageView, CheckBox, RadioButton). A ViewGroup is a container that holds other views (LinearLayout, ConstraintLayout).

6.4 Zaroori attributes (har view ke saath aate hain)

Attribute	Matlab	Example
android:id	View ka naam taake Kotlin usay dhoond sake.	android:id="@+id/btnSubmit"
layout_width / layout_height	Chauraai aur lambai. Teen tareeqe: match_parent, wrap_content, ya number (dp).	android:layout_width="match_parent"
match_parent	Maujooda poori jagah le lo (jitni parent ke paas hai).	Poori row ki chauraai.
wrap_content	Sirf utni jagah lo jitni andar ke content ko chahiye.	Text jitna lamba, button utna chauda.
android:text	View par kya likha dikhega.	android:text="Submit"
android:textSize	Text ka size. **sp** mein.	android:textSize="18sp"
dp	Density-independent pixels. Width, height, margin, padding ke liye.	16dp
sp	Scale-independent pixels. Sirf text size ke liye.	18sp
Margin	View ke **bahar** ki jagah (doosron se faasla).	android:layout_margin="16dp"
Padding	View ke **andar** ki jagah (content aur kinare ke beech).	android:padding="12dp"

Yaad rakho: Margin = outside space (bahar). Padding = inside space (andar). Layout dimensions = dp. Text size = sp. match_parent = poori jagah. wrap_content = zaroorat jitni.

Real-life example: Ek button socho jaise ek dabba. Padding dabbe ke andar sponge hai (content aur deewar ke beech). Margin dabbe ke bahar table par faasla hai (doosre dabbon se).

6.5 Har View: Definition, XML, Kotlin

6.5.1 TextView

TextView screen par **likha hua text dikhata hai**, jaise board par naam likha ho. User ismein kuch likh nahi sakta.

Definition (paper mein likho):
TextView is a UI element that displays text to the user.

Syntax:

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Student Registration"
    android:textSize="24sp"
    android:textStyle="bold" />
```

Example:

```
// Kotlin: find and change text
val tvTitle = findViewById<TextView>(R.id.tvTitle)
tvTitle.text = "Welcome to Android"
```

6.5.2 EditText

EditText ek **khali dibba** hai jis mein user kuch likhta hai, jaise form ki line par naam likhna. **hint** halki si hidayat hoti hai (jaise 'Enter your name') jo likhte hi gayab ho jati hai. **inputType** keyboard ki qisam tay karta hai.

Definition (paper mein likho):
EditText allows the user to enter and edit text. The hint shows a temporary instruction and inputType configures the keyboard and the allowed input.

Syntax:

```
<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name"
    android:inputType="textPersonName" />
```

Example:

```
val etName = findViewById<EditText>(R.id.etName)
val studentName = etName.text.toString()
```

Yaad rakho: EditText se text lene ke liye hamesha .text.toString() likhna hai. Aam inputType: textPersonName (naam), textEmailAddress (email), textPassword (password), number (number), phone (phone number).

6.5.3 Button

Button ek **dabane wali cheez** hai, jaise ghanti ka button. Dabao to kuch hota hai. Kya hona hai, yeh Kotlin mein **setOnClickListener** se likhte hain, yaani 'jab user is par click kare to yeh code chalao'.

Definition (paper mein likho):
A Button is an interactive UI element that the user can click or tap to perform an action.

Syntax:

```
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Submit" />
```

Example:

```
val btnSubmit = findViewById<Button>(R.id.btnSubmit)
btnSubmit.setOnClickListener {
    // code here runs when the button is clicked
}
```

Yaad rakho: Button click event sab se important concept hai. 'btn.setOnClickListener { code }' ka matlab: jab user click kare to yeh code chalao.

6.5.4 ImageView

ImageView screen par **tasveer** dikhata hai (logo, profile photo, product). Tasveer ko pehle **res/drawable** folder mein rakhna parta hai.

Definition (paper mein likho):
ImageView is used to display an image in an Android application.

Syntax:

```
<ImageView
    android:id="@+id/imgLogo"
    android:layout_width="120dp"
    android:layout_height="120dp"
    android:src="@drawable/ic_launcher_foreground"
    android:contentDescription="Application logo" />
```

Example:

```
val image = findViewById<ImageView>(R.id.imgLogo)
image.setImageResource(R.drawable.ic_launcher_foreground)
```

6.5.5 CheckBox

CheckBox ek **chhota dibba** hai jis mein tick lagta hai. Kayi dibbe ek saath tick ho sakte hain. Jaise form mein 'aap ke shauq': Reading, Music, Sports, aap teeno chun sakte ho.

Definition (paper mein likho):
A CheckBox allows the user to select or unselect an option. A user can select multiple checkboxes.

Syntax:

```
<CheckBox
    android:id="@+id/cbJava"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Java" />
```

Example:

```
val cbJava = findViewById<CheckBox>(R.id.cbJava)
if (cbJava.isChecked) {
    println("Java selected")
}
```

Yaad rakho: Tick hai ya nahi, yeh .isChecked (true/false) batata hai.

6.5.6 RadioButton aur RadioGroup

RadioButton mein **sirf ek** option chun sakte hain, jaise MCQ mein ek jawab. Gender: Male ya Female, dono ek saath nahi. Kayi RadioButtons ko ek **RadioGroup** mein band karna parta hai, tab hi 'sirf ek chuna jaye' wala usool chalta hai.

Definition (paper mein likho):
A RadioButton is used when the user must select only one option from a group. RadioButtons are placed inside a RadioGroup so that only one can be selected at a time.

Syntax:

```
<RadioGroup
    android:id="@+id/rgGender"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <RadioButton
        android:id="@+id/rbMale"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Male" />
    <RadioButton
        android:id="@+id/rbFemale"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Female" />
</RadioGroup>
```

Example:

```
val rbMale = findViewById<RadioButton>(R.id.rbMale)
val gender = if (rbMale.isChecked) "Male" else "Female"
```

Yaad rakho: CheckBox = kayi chun sakte hain. RadioButton = sirf ek. RadioButton ke liye RadioGroup lazmi.

6.5.7 Switch

Switch bijli ke **on/off** switch ki tarah hai. Notifications, Dark mode, Wi-Fi, Sound on ya off karne ke liye.

Definition (paper mein likho):
A Switch is used to turn an option ON or OFF.

Syntax:

```
<Switch
    android:id="@+id/switchNotify"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Notifications" />
```

Example:

```
val switchNotify = findViewById<Switch>(R.id.switchNotify)
val status = if (switchNotify.isChecked) "ON" else "OFF"
```

6.5.8 Spinner

Spinner ek **drop-down list** hai. Pehle sirf ek line dikhti hai, click karo to list khulti hai (Pakistan, Qatar, UAE, Saudi Arabia), aur aap ek chunte ho. Jab bohat saare options hon tab use hota hai.

Definition (paper mein likho):
A Spinner provides a drop-down list from which the user can select one option.

Syntax:

```
<Spinner
    android:id="@+id/spCountry"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Example:

```
val spCountry = findViewById<Spinner>(R.id.spCountry)
val countries = arrayOf("Pakistan", "Qatar", "UAE", "Saudi Arabia")
val adapter = ArrayAdapter(this,
    android.R.layout.simple_spinner_item, countries)
adapter.setDropDownViewResource(
    android.R.layout.simple_spinner_dropdown_item)
spCountry.adapter = adapter
val selected = spCountry.selectedItem.toString()
```

Yaad rakho: Spinner ko list dene ke liye 'adapter' chahiye (adapter list aur spinner ke beech pul hai).

6.5.9 ProgressBar

ProgressBar dikhata hai ke **kaam ho raha hai**, jaise 'Loading...' ka ghoomta hua gola. Data load hote waqt ya file download hote waqt.

Definition (paper mein likho):
A ProgressBar shows that a task is currently in progress.

Syntax:

```
<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

Example:

```
val progressBar = findViewById<ProgressBar>(R.id.progressBar)
progressBar.visibility = View.VISIBLE   // show
progressBar.visibility = View.GONE      // hide
```

Part 7: Complete Apps: Screen + XML + Kotlin (paper ka sab se bara sawal)

Sawal aise aata hai: 'Yeh screen banao aur uska XML aur Kotlin code likho'

Isay **4 qadam** mein hal karo, har baar:

Qadam	Kya karna hai
1. Screen ko dekho	Kaun kaun si cheezein hain? (TextView, EditText, Button...) Unhein upar se neeche likh lo.
2. XML likho	Root mein LinearLayout (vertical). Har view ke liye tag, **id**, width, height, text. Id ka naam yaad rakho.
3. Kotlin likho	onCreate mein **setContentView**, phir har view ko **findViewById** se pakro, button par **setOnClickListener**, andar logic, aakhir mein result TextView par dikhao.
4. Output batao	Ek line mein likho ke user kya karega aur screen par kya aayega.

Hamesha ka pattern (Kotlin ka dhancha):

```
package com.example.myapp

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)   // XML jodo

        val etName = findViewById<EditText>(R.id.etName)   // views pakro
        val btnSubmit = findViewById<Button>(R.id.btnSubmit)
        val tvResult = findViewById<TextView>(R.id.tvResult)

        btnSubmit.setOnClickListener {           // click par
            val name = etName.text.toString()    // input lo
            tvResult.text = "Welcome, $name!"    // result dikhao
        }
    }
}
```

Yaad rakho: Har app mein yehi 5 cheezein hoti hain: setContentView, findViewById, setOnClickListener, input padhna (.text.toString() ya .isChecked), aur result TextView par dikhana. Import lines bhi likhna mat bhoolna (har view ka alag import).

App 1: Hello User (EditText + Button + TextView)

Screen: ek title, naam likhne ki jagah (EditText), ek Submit button, aur neeche result. User naam likhe, Submit dabaye, to 'Welcome, Ali!' dikhe.

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello User App"
        android:textSize="24sp" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name"
        android:inputType="textPersonName" />

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit" />

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.helloapp

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etName = findViewById<EditText>(R.id.etName)
        val btnSubmit = findViewById<Button>(R.id.btnSubmit)
        val tvResult = findViewById<TextView>(R.id.tvResult)

        btnSubmit.setOnClickListener {
            val name = etName.text.toString()
            tvResult.text = "Welcome, $name!"
        }
    }
}
```

Kaise chalta hai: User naam likhta hai (EditText) -> Submit dabata hai -> setOnClickListener chalta hai -> naam padha jata hai -> Kotlin text banata hai -> TextView par dikhta hai. **Output:** user ne Ali likha to 'Welcome, Ali!'.

App 2: Simple Calculator (2 numbers + 4 buttons)

Screen: do number likhne ki jagah, 4 buttons (+, -, *, /) ek hi line mein, aur result. Buttons ek hi line mein barabar jagah lein, is liye horizontal LinearLayout aur **layout_weight=1**, **width=0dp**.

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etNum1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="First number"
        android:inputType="numberDecimal" />

    <EditText
        android:id="@+id/etNum2"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Second number"
        android:inputType="numberDecimal" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnAdd"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="+" />
        <Button
            android:id="@+id/btnSub"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="-" />
        <Button
            android:id="@+id/btnMul"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="*" />
        <Button
            android:id="@+id/btnDiv"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="/" />
    </LinearLayout>

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Result:"
        android:textSize="22sp" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.calculatorapp

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etNum1 = findViewById<EditText>(R.id.etNum1)
        val etNum2 = findViewById<EditText>(R.id.etNum2)
        val tvResult = findViewById<TextView>(R.id.tvResult)

        fun calculate(op: Char) {
            val n1 = etNum1.text.toString().toDoubleOrNull()
            val n2 = etNum2.text.toString().toDoubleOrNull()
            if (n1 == null || n2 == null) {
                tvResult.text = "Please enter both numbers"
                return
            }
            val result = when (op) {
                '+' -> n1 + n2
                '-' -> n1 - n2
                '*' -> n1 * n2
                else -> if (n2 == 0.0) null else n1 / n2
            }
            tvResult.text = if (result == null) "Cannot divide by zero"
                            else "Result = $result"
        }

        findViewById<Button>(R.id.btnAdd).setOnClickListener { calculate('+') }
        findViewById<Button>(R.id.btnSub).setOnClickListener { calculate('-') }
        findViewById<Button>(R.id.btnMul).setOnClickListener { calculate('*') }
        findViewById<Button>(R.id.btnDiv).setOnClickListener { calculate('/') }
    }
}
```

Samjho: toDoubleOrNull() text ko number banata hai, agar khali ya ghalat ho to null deta hai (crash nahi). when se operator ke mutabiq hisaab. **Output:** 10 aur 5 par '+' dabao to Result = 15.0.

App 3: CheckBox app (programming languages ki expertise)

Sawal (slide se): C++, Java, PHP ke teen CheckBox banao, ek button dabane par tick kiye hue languages TextView mein dikhao.

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/txtOutput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Output Here"
        android:textSize="20sp"
        android:textStyle="bold"
        android:gravity="center"
        android:layout_marginBottom="24dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Experts:"
        android:textSize="18sp" />

    <CheckBox
        android:id="@+id/cbCPP"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="C++" />
    <CheckBox
        android:id="@+id/cbJava"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Java" />
    <CheckBox
        android:id="@+id/cbPhp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="PHP" />

    <Button
        android:id="@+id/btnShow"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Show Experts" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.expertselectorapp

import android.os.Bundle
import android.widget.Button
import android.widget.CheckBox
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val txtOutput = findViewById<TextView>(R.id.txtOutput)
        val cbCPP = findViewById<CheckBox>(R.id.cbCPP)
        val cbJava = findViewById<CheckBox>(R.id.cbJava)
        val cbPhp = findViewById<CheckBox>(R.id.cbPhp)
        val btnShow = findViewById<Button>(R.id.btnShow)

        btnShow.setOnClickListener {
            val experts = mutableListOf<String>()
            if (cbCPP.isChecked) experts.add("C++")
            if (cbJava.isChecked) experts.add("Java")
            if (cbPhp.isChecked) experts.add("PHP")

            txtOutput.text = if (experts.isEmpty()) {
                "Please select at least one expert."
            } else {
                "Selected Experts:\n" + experts.joinToString("\n")
            }
        }
    }
}
```

Samjho: ek khali list (mutableListOf) banai, har tick wale language ko usme add kiya (isChecked), aakhir mein joinToString se sab ko TextView mein likh diya. **Note:** slide ke XML mein button par android:onClick bhi likha tha, aur Kotlin mein setOnClickListener bhi. Dono ek saath nahi lagate, ek hi tareeqa rakho (setOnClickListener behtar hai).

App 4: User Preferences (RadioButton + CheckBox + Switch)

Screen: Gender (RadioButtons Male/Female), Hobbies (CheckBoxes Reading, Music, Sports), Notifications (Switch), ek button SHOW PREFERENCES, aur result jahan teeno dikhein.

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="User Preferences"
        android:textSize="22sp"
        android:textStyle="bold"
        android:gravity="center"
        android:layout_marginBottom="16dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Gender:"
        android:textStyle="bold" />

    <RadioGroup
        android:id="@+id/radioGroupGender"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">
        <RadioButton
            android:id="@+id/rbMale"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Male"
            android:checked="true" />
        <RadioButton
            android:id="@+id/rbFemale"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Female" />
    </RadioGroup>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Hobbies:"
        android:textStyle="bold" />

    <CheckBox
        android:id="@+id/cbReading"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Reading" />
    <CheckBox
        android:id="@+id/cbMusic"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Music" />
    <CheckBox
        android:id="@+id/cbSports"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sports" />

    <Switch
        android:id="@+id/switchNotify"
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:text="Enable Notifications:"
        android:checked="true" />

    <Button
        android:id="@+id/btnShow"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="SHOW PREFERENCES"
        android:layout_marginTop="8dp" />

    <TextView
        android:id="@+id/txtResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Result:"
        android:textSize="14sp"
        android:padding="12dp"
        android:layout_marginTop="8dp" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.preferencesapp

import android.os.Bundle
import android.widget.Button
import android.widget.CheckBox
import android.widget.RadioButton
import android.widget.Switch
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rbMale = findViewById<RadioButton>(R.id.rbMale)
        val rbFemale = findViewById<RadioButton>(R.id.rbFemale)
        val cbReading = findViewById<CheckBox>(R.id.cbReading)
        val cbMusic = findViewById<CheckBox>(R.id.cbMusic)
        val cbSports = findViewById<CheckBox>(R.id.cbSports)
        val switchNotify = findViewById<Switch>(R.id.switchNotify)
        val btnShow = findViewById<Button>(R.id.btnShow)
        val txtResult = findViewById<TextView>(R.id.txtResult)

        btnShow.setOnClickListener {
            val gender = when {
                rbMale.isChecked -> "Male"
                rbFemale.isChecked -> "Female"
                else -> "Not selected"
            }
            val hobbies = mutableListOf<String>()
            if (cbReading.isChecked) hobbies.add("Reading")
            if (cbMusic.isChecked) hobbies.add("Music")
            if (cbSports.isChecked) hobbies.add("Sports")
            val hobbyText = if (hobbies.isEmpty()) "None"
                            else hobbies.joinToString(", ")
            val notifications = if (switchNotify.isChecked) "ON" else "OFF"

            txtResult.text = "Result:\nGender: $gender\n" +
                "Hobbies: $hobbyText\nNotifications: $notifications"
        }
    }
}
```

Output: Result: Gender: Male, Hobbies: Music, Sports, Notifications: ON. **Note:** slide ke code mein dono RadioButtons par checked=true likha tha. RadioGroup mein sirf ek hi checked rakho (humne Male rakha).

App 5: Spinner + ProgressBar (country chunna)

Screen: ek drop-down (Spinner) mein mulk, ek button, ek ghoomta hua ProgressBar, aur result.

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <Spinner
        android:id="@+id/spCountry"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/btnShow"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Country" />

    <ProgressBar
        android:id="@+id/progressBar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:visibility="gone" />

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.spinnerapp

import android.os.Bundle
import android.view.View
import android.widget.ArrayAdapter
import android.widget.Button
import android.widget.ProgressBar
import android.widget.Spinner
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val spCountry = findViewById<Spinner>(R.id.spCountry)
        val btnShow = findViewById<Button>(R.id.btnShow)
        val progressBar = findViewById<ProgressBar>(R.id.progressBar)
        val tvResult = findViewById<TextView>(R.id.tvResult)

        val countries = arrayOf("Pakistan", "Qatar", "UAE", "Saudi Arabia")
        val adapter = ArrayAdapter(this,
            android.R.layout.simple_spinner_item, countries)
        adapter.setDropDownViewResource(
            android.R.layout.simple_spinner_dropdown_item)
        spCountry.adapter = adapter

        btnShow.setOnClickListener {
            progressBar.visibility = View.VISIBLE
            tvResult.text = "Selected: ${spCountry.selectedItem}"
            progressBar.visibility = View.GONE
        }
    }
}
```

App 6: Login screen + Intent (do activities)

Screen 1 (Login): username, password, Login button. **Screen 2 (Welcome):** 'Welcome, Ali!'. Login dabane par doosri screen khule aur naam saath jaye. Is ke liye **Intent** chahiye (Part 10 mein detail). **Bohat zaroori:** har nayi Activity ko **AndroidManifest.xml** mein bhi likhna parta hai, warna app crash ho jati hai.

activity_main.xml (Login screen):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etUser"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username"
        android:inputType="textPersonName" />

    <EditText
        android:id="@+id/etPass"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.loginapp

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etUser = findViewById<EditText>(R.id.etUser)
        val etPass = findViewById<EditText>(R.id.etPass)
        val btnLogin = findViewById<Button>(R.id.btnLogin)

        btnLogin.setOnClickListener {
            val user = etUser.text.toString()
            val pass = etPass.text.toString()
            if (user.isEmpty() || pass.isEmpty()) {
                Toast.makeText(this, "Please fill all fields",
                    Toast.LENGTH_SHORT).show()
            } else {
                val intent = Intent(this, WelcomeActivity::class.java)
                intent.putExtra("USERNAME", user)
                startActivity(intent)
            }
        }
    }
}
```

activity_welcome.xml (Welcome screen):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center">

    <TextView
        android:id="@+id/tvWelcome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="24sp" />
</LinearLayout>
```

WelcomeActivity.kt:

```
package com.example.loginapp

import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class WelcomeActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_welcome)

        val tvWelcome = findViewById<TextView>(R.id.tvWelcome)
        val user = intent.getStringExtra("USERNAME")
        tvWelcome.text = "Welcome, ${user ?: "User"}!"
    }
}
```

AndroidManifest.xml (navi activity ka entry):

```
<application ...>
    <activity android:name=".MainActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <activity android:name=".WelcomeActivity" />   <!-- zaroor likho -->
</application>
```

Toast ek chhota sa message hai jo kuch der ke liye neeche dikhta hai aur khud ghayab ho jata hai. **Flow:** Login click -> input check -> Intent banaya -> putExtra se naam rakha -> startActivity -> WelcomeActivity khuli -> getStringExtra se naam nikala -> TextView par dikhaya.

Part 8: Layouts: LinearLayout, RelativeLayout, ConstraintLayout

8.1 Layout kya hota hai? (pehle yeh samjho)

Ek class mein bachon ko baithana hai. Aap kai tareeqon se bitha sakte ho: ek line mein (Linear), ek doosre ke 'bagal mein, neeche, upar' (Relative), ya har bachay ko **rassi se bandh kar** deewar aur doosre bachon se jod dena (Constraint). Layout wohi **baithane ka tareeqa** hai: screen par views (button, text) kahan aur kaise rakhe jayein.

Definition (paper mein likho):
A layout is a ViewGroup that defines the structure and arrangement of the views (UI elements) on the screen. Common layouts are LinearLayout, RelativeLayout and ConstraintLayout.

8.2 LinearLayout

Bachay **ek line** mein khade hain. Ya to **ek ke neeche ek** (vertical) ya **ek ke bagal mein ek** (horizontal). Sab se aasan layout hai. Direction **android:orientation** se tay hoti hai.

Definition (paper mein likho):
LinearLayout is a ViewGroup that arranges its child views in a single direction, either vertically or horizontally, according to the orientation attribute.

Syntax:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">   <!-- or horizontal -->
    ...child views...
</LinearLayout>
```

Example:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="24sp" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login" />
</LinearLayout>
```

Yaad rakho: Views upar se neeche tarteeb se aate hain jis tarteeb mein XML mein likhe hon.

Attribute	Matlab
android:orientation	vertical = ek ke neeche ek. horizontal = ek ke bagal mein ek.
android:layout_weight	Bachi hui jagah ko views mein **hissa baant** kar dena. Weight 1 aur 1 = aadhi aadhi. Is ke saath width ko **0dp** likhna (horizontal ho to).
android:gravity	View ke **andar ki cheezon** ko kahan rakhna hai (center, left, right).
android:layout_gravity	Yeh view khud **apne parent mein** kahan baithe.
padding / margin	Andar / bahar ki jagah.

layout_weight ka example: do buttons barabar chaure karne hain:

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <Button
        android:layout_width="0dp"
```

```
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Yes" />
    <Button
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="No" />
</LinearLayout>
```

8.3 RelativeLayout

Bachay **ek doosre ke hawale se** baithte hain: 'main Ali ke neeche hoon', 'main Sara ke daayen (right) mein hoon', 'main poore class ke darmiyan mein hoon'. Har view ki position **dusre view ya parent ke relative** batai jati hai. Is mein view ko dusre view ki **id** se refer karte hain.

Definition (paper mein likho):
RelativeLayout is a ViewGroup in which the position of each child view is specified relative to sibling views or to the parent.

Syntax:

```
<RelativeLayout ...>
    <View android:layout_below="@id/otherView" />
</RelativeLayout>
```

Example:

```
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Student Form"
        android:layout_centerHorizontal="true" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Name"
        android:layout_below="@id/tvTitle" />

    <Button
        android:id="@+id/btnSave"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Save"
        android:layout_below="@id/etName"
        android:layout_alignParentRight="true" />
</RelativeLayout>
```

Yaad rakho: Title upar beech mein, EditText title ke neeche, Save button EditText ke neeche aur screen ke right kinare par.

Attribute	Matlab
layout_below	Is view ke **neeche**.
layout_above	Is view ke **upar**.

Attribute	Matlab
layout_toRightOf / layout_toLeftOf	Is view ke **daayen / baayen**.
layout_alignParentTop / Bottom / Left / Right	Parent ke upar / neeche / left / right **kinare** par chipak jao.
layout_centerInParent	Parent ke **bilkul beech** mein.
layout_centerHorizontal / centerVertical	Sirf **afuqi (horizontal)** ya **amudi (vertical)** beech mein.
layout_alignLeft / alignTop	Doosre view ke left/top ke saath **line mein**.

8.4 ConstraintLayout

Har bachay ko **rassiyon (constraints)** se bandhte hain: ek rassi upar wali deewar se, ek neeche wale bachay se, ek left kinare se. Rassiyon ki wajah se bachay jaha chaho wahin rehte hain, aur screen chhoti ho ya badi, **sab theek jagah** rehte hain. Yeh sab se modern aur **Android ki recommended** layout hai, aur design editor mein drag-drop se bhi banti hai.

Definition (paper mein likho):
ConstraintLayout is a flexible ViewGroup that positions views by defining constraints (connections) between views and to the parent or guidelines. It supports flat view hierarchies and responsive layouts.

Syntax:

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto" ...>
    <View
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Example:

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome"
        android:textSize="24sp"
        android:layout_marginTop="32dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Enter name"
        android:layout_marginTop="24dp"
        android:layout_marginStart="16dp"
        android:layout_marginEnd="16dp"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button
        android:id="@+id/btnOk"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="OK"
        android:layout_marginTop="16dp"
        app:layout_constraintTop_toBottomOf="@id/etName"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Yaad rakho: Har view ko kam az kam ek **horizontal** (Start/End) aur ek **vertical** (Top/Bottom) constraint dena zaroori hai, warna view corner mein chala jata hai.

Attribute	Matlab
app:layout_constraintTop_toTopOf	Mere upar ka kinara = X ke upar ke kinare ke saath.
app:layout_constraintTop_toBottomOf	Mere upar ka kinara = X ke **neeche** ke saath (X ke neeche aa jao).
app:layout_constraintBottom_toBottomOf	Mera neeche ka kinara = X ke neeche ke saath.
app:layout_constraintStart_toStartOf	Mera shuru (left) = X ke shuru ke saath.
app:layout_constraintEnd_toEndOf	Mera aakhir (right) = X ke aakhir ke saath.
parent	Poori layout (screen) ko X bana do.
0dp (match constraint)	Width/height 0dp ka matlab 'constraints ke darmiyan poori jagah bharo'.
xmlns:app	Namespace. 'app:' wale attributes ke liye lazmi.

8.5 Teeno layouts ka muqabla (table zaroor banao)

Pehlu	LinearLayout	RelativeLayout	ConstraintLayout
Position kaise	Ek line (vertical ya horizontal)	Doosre views / parent ke relative	Constraints (rassiyon) se
Seekhna	Sab se aasan	Darmiyana	Thora mushkil, magar tools ki madad
Complex design	Kamzor (nesting chahiye)	Theek	Behtareen
Nested layouts	Zyada (slow ho sakta hai)	Kam	Bohat kam (flat hierarchy)
Mashhoor attribute	orientation, layout_weight	layout_below, alignParentRight	layout_constraintTop_toTopOf
Kab use karein	Chhoti form, ek line	Purani apps, simple relative position	Modern apps, bari/complex screens

Yaad rakho: Choti simple screen = LinearLayout. Complex responsive screen = ConstraintLayout (recommended). RelativeLayout purani hai. Teeno ViewGroup hain.

Do aur layouts (sirf naam): FrameLayout (views ek ke upar ek, jaise tasveer par text), **TableLayout** (rows aur columns ka jadwal).

Part 9: Activity aur Activity Lifecycle (7 methods)

9.1 Activity kya hai?

Aap ne app kholi to ek **screen** nazar aati hai: Login screen, Home screen, Profile screen. Har aise ek screen ko Android mein **Activity** kehte hain. Jaise kitaab ka ek page. App mein kayi pages (Activities) hoti hain.

Definition (paper mein likho):
An Activity represents a single screen with a user interface in an Android application. Examples: Login screen (LoginActivity), Home screen (MainActivity), Profile screen (ProfileActivity).

Syntax:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

9.2 Activity Lifecycle kya hai?

Insaan ki zindagi mein paidaish, bachpan, jawani, budhapa aur maut aati hai. Activity ki bhi aisi hi **zindagi** hoti hai: paida hoti hai, nazar aati hai, kaam karti hai, ruk jati hai, aur khatam hoti hai. Har marhale par Android ek khaas method (callback) bulata hai. Yeh method hum override kar ke apna kaam likhte hain.

Definition (paper mein likho):
The Android Activity Lifecycle is the sequence of states through which an Activity passes from creation to destruction. Android calls lifecycle callback methods when the state changes: onCreate(), onStart(), onResume(), onPause(), onStop(), onRestart() and onDestroy().

Lifecycle ka flow (diagram)



onRestart(): jab ruki hui (stopped) Activity dobara user ke saamne aati hai to **onStop() ke baad onRestart() -> onStart() -> onResume()** chalte hain. Yaani wapsi ka raasta: onStop -> onRestart -> onStart -> onResume.

Yaad rakho: Zindagi ka silsila: onCreate -> onStart -> onResume (RUNNING) -> onPause -> onStop -> onDestroy. Wapas aane par: onRestart -> onStart -> onResume.

9.3 Saare 7 methods: Definition, Purpose, Usage

1. onCreate()

Paidaish. Jab Activity pehli baar banti hai, yeh sab se pehle chalta hai. Jaise ghar ka naksha bana kar pehli baar ghar khada karna. Yahan hum screen ka design lagate hain (setContentView), variables banate hain, buttons tayyar karte hain, shuruati data load karte hain.

Definition (paper mein likho):
onCreate() is called when the Activity is first created. It is used to create the user interface, initialize variables, set up buttons and load initial data.

Syntax:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

Example:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    val btn = findViewById<Button>(R.id.btnShow)
    btn.setOnClickListener { /* action */ }
    println("onCreate() called")
}
```

Yaad rakho: Yeh sirf **ek baar** chalta hai jab Activity banti hai. UI yahin set hota hai.

2. onStart()

Nazar aana shuru. Activity ab screen par **dikhne lagi hai**, magar abhi user us se kaam nahi kar sakta. Jaise parda hata, stage nazar aane laga, magar actors abhi tayyar nahi. onCreate ke foran baad, ya onRestart ke baad chalta hai. Jab user ek app se doosri par jata aur wapas aata hai (jaise notification dabane par) tab bhi.

Definition (paper mein likho):
onStart() is called when the Activity becomes visible to the user. It is called after onCreate() or after onRestart().

Syntax:

```
override fun onStart() {
    super.onStart()
}
```

Example:

```
override fun onStart() {
    super.onStart()
    println("onStart() called: Activity is now visible")
}
```

Yaad rakho: Yahan wo kaam karo jo Activity ke nazar aane par chahiye.

3. onResume()

Kaam shuru. Activity ab **foreground** mein hai aur user us se **interact** kar sakta hai (button dabana, likhna). Isay 'Activity running' bhi kehte hain. Yeh onStart ke baad hamesha chalta hai, aur onPause ke baad wapas aane par bhi. Jaise stage par actors tayyar aur show shuru.

Definition (paper mein likho):
onResume() is called when the Activity is in the foreground and the user can interact with it. At this point the Activity is running.

Syntax:

```
override fun onResume() {
    super.onResume()
}
```

Example:

```
override fun onResume() {
    super.onResume()
    println("onResume() called: Activity is active")
    // start animations, camera, music controls
}
```

Yaad rakho: Yahan wo cheezein dobara shuru karo jo onPause mein rok di thin (camera, animation, music controls).

4. onPause()

Focus jaa raha hai. Koi aur cheez aage aa gayi, jaise phone call aa gayi, ya koi dialog khul gaya, ya aap ne Home ya Back dabaya. Activity abhi thori nazar aa sakti hai, magar user usay chala nahi sakta. Jaise class mein koi aa kar bole to aap kitaab thori der ke liye rakh dete ho. **Yeh method jaldi khatam hona chahiye**, bhaari kaam yahan na karo.

Definition (paper mein likho):
onPause() is called when the Activity is losing focus, for example when another Activity comes in front, a dialog appears or a call interrupts. It should be quick.

Syntax:

```
override fun onPause() {
    super.onPause()
}
```

Example:

```
override fun onPause() {
    super.onPause()
    println("onPause() called")
    // pause animation, music, save small unsaved data
}
```

Yaad rakho: Yahan chalte hue kaam ko **rok** do (pause) aur chhota data save kar lo.

5. onStop()

Ab nazar nahi aati. Doosri Activity ne poori screen le li, ya Home dabaya. Jaise aap ek kamray se doosre kamray mein chale gaye, pehla kamra ab nazar nahi aata magar mojood hai. Activity ab **stopped** hai.

Definition (paper mein likho):
onStop() is called when the Activity is no longer visible to the user because another Activity has taken its place.

Syntax:

```
override fun onStop() {
    super.onStop()
}
```

Example:

```
override fun onStop() {
    super.onStop()
    println("onStop() called")
    // release resources that are not needed
}
```

Yaad rakho: Yahan jo cheezein ab zaroori nahi unhein **release** kar do (camera, sensors, location, bhaari resources).

6. onRestart()

Wapsi. Jo Activity ruki (stopped) thi woh dobara user ke saamne aa rahi hai. Jaise aap doosre kamray se wapas pehle kamray mein aaye. **Pehli baar app khulne par yeh kabhi nahi chalta**, sirf stop hone ke baad wapas aane par chalta hai.

Definition (paper mein likho):
onRestart() is called when a stopped Activity is about to become visible again. It is never called when the Activity is started for the first time.

Syntax:

```
override fun onRestart() {
    super.onRestart()
}
```

Example:

```
override fun onRestart() {
    super.onRestart()
    println("onRestart() called")
}
```

Yaad rakho: onRestart ke baad foran onStart aur phir onResume chalte hain.

7. onDestroy()

Khatam. Activity band ho rahi hai. Yeh tab hota hai jab Activity finish ki jaye, ya Android usay dobara bana raha ho (jaise phone ghumane par, ya memory kam ho). Aakhri safai yahin hoti hai. **Important:** sirf onDestroy mein zaroori data save karne par bharosa na karo, kyunke yeh hamesha guarantee se nahi chalta.

Definition (paper mein likho):
onDestroy() is called when the Activity is being destroyed or closed. It is used for final cleanup.

Syntax:

```
override fun onDestroy() {
    super.onDestroy()
}
```

Example:

```
override fun onDestroy() {
    super.onDestroy()
    println("onDestroy() called: final cleanup")
}
```

Yaad rakho: Aakhri cleanup yahan. Zaroori data pehle (onPause/onStop mein) save karo.

9.4 Saare 7 methods ek table mein (paper ke liye)

Method	Kab chalta hai	Usage (kya karna hai)
onCreate()	Activity pehli baar banti hai	UI set karna, variables, buttons, initial data
onStart()	Activity nazar aane lagti hai	Visible hone par kaam
onResume()	Foreground, user interact kar sakta hai (running)	Animation, camera, music dobara shuru
onPause()	Focus kho rahi hai (dialog, call, Home, Back)	Kaam rokna, chhota data save, jaldi khatam
onStop()	Bilkul nazar nahi aa rahi	Bhaari resources release
onRestart()	Stopped Activity wapas aa rahi hai	Dobara shuru karne ki tayyari
onDestroy()	Activity khatam ho rahi hai	Final cleanup

9.5 Alag alag soorton mein kaun se methods chalte hain

Soorat	Methods ka silsila
App pehli baar kholi	onCreate -> onStart -> onResume
Home button dabaya	onPause -> onStop
Home se wapas app khol li	onRestart -> onStart -> onResume
Doosri Activity (poori screen) khuli	Pehli Activity: onPause -> onStop. Doosri: onCreate -> onStart -> onResume.
Phone call / dialog aa gaya (thora sa ooper)	onPause (agar poori chhup gayi to onStop bhi)
Back button dabaya (Activity band)	onPause -> onStop -> onDestroy

9.6 Lifecycle kyun zaroori hai?

Kyunke app kabhi **foreground** mein hoti hai aur kabhi **background** mein. Lifecycle ko theek se sambhalne se: (1) **Resources bachte hain**: jab screen nazar na aa rahi ho to be-faida kaam rok do. (2) **Performance behtar** hoti hai. (3) **UI state** wapas aane par theek ho jati hai. (4) **Hardware sahi chalta hai**: camera, sensors, location, media. (5) **User ka tajurba achha** hota hai: koi ajeeb behavior ya crash nahi.

Real-life example: Music Player. onResume mein music controls active. onPause mein gaana pause ya aawaz kam. onStop mein ghair-zaroori resources chhor do. Wapas aane par onRestart -> onStart -> onResume: controls dobara active.

9.7 Lab app: Activity ki halat screen par dikhana

Maqsad: screen par likha aaye ke Activity ab kis halat mein hai. Sirf onCreate, onStart, onResume mein TextView badalte hain, kyunke onPause/onStop/onDestroy par screen nazar nahi aati.

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/txtTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Android Activity Lifecycle"
        android:textSize="26sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/txtStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="30dp"
        android:text="Activity Status"
        android:textSize="20sp"
        android:gravity="center" />
</LinearLayout>
```

MainActivity.kt:

```
package com.example.activitylifecycledemo

import android.os.Bundle
import android.util.Log
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    private lateinit var txtStatus: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        txtStatus = findViewById(R.id.txtStatus)
        txtStatus.text = "onCreate()\nActivity Created"
        Log.d("Lifecycle", "onCreate")
    }
    override fun onStart() {
        super.onStart()
        txtStatus.text = "onStart()\nActivity Visible"
        Log.d("Lifecycle", "onStart")
    }
    override fun onResume() {
        super.onResume()
        txtStatus.text = "onResume()\nActivity Active"
        Log.d("Lifecycle", "onResume")
    }
    override fun onPause() {
        super.onPause()
        Log.d("Lifecycle", "onPause")
    }
    override fun onStop() {
        super.onStop()
        Log.d("Lifecycle", "onStop")
    }
    override fun onRestart() {
        super.onRestart()
        Log.d("Lifecycle", "onRestart")
    }
    override fun onDestroy() {
        super.onDestroy()
        Log.d("Lifecycle", "onDestroy")
    }
}
```

Test kaise karein: (1) App chalao: 'onResume() Activity Active' dikhta hai. (2) Home dabao: onPause -> onStop. (3) Wapas app kholo: onRestart -> onStart -> onResume. (4) Back dabao: onDestroy. Log.d ke messages Android Studio ke **Logcat** mein dikhte hain, jahan saare 7 methods nazar aate hain.

Part 10: Intent (sirf types aur methods)

10.1 Intent kya hai?

Intent ek **paigham (message)** hai jo Android ko kehta hai: 'yeh kaam karwa do'. Jaise aap school mein monitor se kehte ho 'teacher ko bulao'. Intent se hum ek screen se doosri screen khol sakte hain, data bhej sakte hain, website kholna, phone dialer kholna, message share karna.

Definition (paper mein likho):
An Intent is a messaging object that requests an action from another Android component, such as an Activity or a system application. It is used to start Activities, pass data, open web pages, launch the dialer and share content.

Syntax:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

Yaad rakho: Syntax samjho: Intent(...) paigham banata hai. this = maujooda Activity. SecondActivity::class.java = manzil. startActivity(intent) = paigham bhej do. Import: android.content.Intent

10.2 Intent ki 2 types

Type 1: Explicit Intent

Isme aap **sahi pata** likhte ho: 'Ali ke ghar jao'. Yaani aap Android ko **seedha batate ho** ke kaunsi Activity kholni hai. Aam tor par apni hi app ki ek screen se doosri screen kholne ke liye.

Definition (paper mein likho):
An Explicit Intent specifies the exact component (Activity) that should handle the request. It is mostly used to move between Activities of the same application.

Syntax:

```
val intent = Intent(this, TargetActivity::class.java)
startActivity(intent)
```

Example:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)   // MainActivity -> SecondActivity
```

Yaad rakho: Fayde: manzil seedhi pata, in-app navigation ke liye behtar, data bhejna aasan.

Type 2: Implicit Intent

Isme aap **pata nahi, sirf kaam** batate ho: 'kisi ko bulao jo gaana chala sake'. Aap sirf **action** batate ho (website kholo), aur Android khud dhoondta hai kaunsi app yeh kar sakti hai (browser). Agar kayi apps hon to chooser dikhata hai. Doosri apps ke saath kaam karne ke liye.

Definition (paper mein likho):
An Implicit Intent describes an action to perform without naming a specific component. Android finds a suitable app that can handle the request.

Syntax:

```
val intent = Intent(Intent.ACTION_VIEW, Uri.parse("url"))
startActivity(intent)
```

Example:

```
val intent = Intent(
    Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com")
)
startActivity(intent)   // opens the browser
```

Yaad rakho: Fayde: doosri apps use kar sakte ho, har feature khud banana nahi parta. Import: android.net.Uri

10.3 Explicit vs Implicit (table zaroor banao)

Pehlu	Explicit Intent	Implicit Intent
Manzil	Seedha batate hain (exact component)	Sirf action batate hain
Component ka naam	Aam tor par dete hain	Aam tor par nahi dete
Kis liye	Apni app ki screens ke beech navigation	Doosri apps se kaam karwana
Example	SecondActivity kholna	Website kholna
Data	Extras (putExtra) se	Extras, data URI, ya dono
Common methods	startActivity()	startActivity(), Intent.createChooser()

Yaad rakho: Dono Intents mein data ja sakta hai. Farq sirf yeh hai ke manzil kaise batai jati hai: naam se (explicit) ya kaam se (implicit).

10.4 Intent ke Important Methods

1. startActivity()

Paigham bhejne wala **button**. Intent bana liya, ab isay chalane ke liye startActivity bulate hain.

Definition (paper mein likho):
startActivity() starts an Activity using the supplied Intent. It is used for navigation to another Activity or for launching an external application.

Syntax:

```
startActivity(intent)
```

Example:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

2. putExtra()

Paigham ke saath **saman (data) rakhna**. Data ek **key-value** jori hota hai, jaise lifafe par naam aur andar khat.

Definition (paper mein likho):
putExtra() adds data (an extra) to an Intent as a key-value pair.

Syntax:

```
intent.putExtra("key", value)
```

Example:

```
val intent = Intent(this, SecondActivity::class.java)
intent.putExtra("STUDENT_NAME", "Ali")
intent.putExtra("AGE", 20)
startActivity(intent)
```

3. getStringExtra() (aur doosre getExtra methods)

Doosri screen mein lifafa **kholna** aur saman nikalna. Key wohi likhni hai jo bhejte waqt likhi thi, warna data nahi milega. Alag data type ke liye alag method.

Definition (paper mein likho):
getStringExtra() retrieves a String value sent with the Intent, using the same key. Other methods exist for other data types.

Syntax:

```
val value = intent.getStringExtra("key")
```

Example:

```
// In SecondActivity
val studentName = intent.getStringExtra("STUDENT_NAME")
textView.text = "Welcome, ${studentName ?: "Student"}!"
val age = intent.getIntExtra("AGE", 0)
```

Data type	Bhejne ka method	Receive karne ka method
String	putExtra("name", "Ali")	getStringExtra("name")
Integer	putExtra("age", 20)	getIntExtra("age", 0)
Boolean	putExtra("active", true)	getBooleanExtra("active", false)
Double	putExtra("marks", 87.5)	getDoubleExtra("marks", 0.0)

Yaad rakho: getIntExtra, getBooleanExtra, getDoubleExtra mein **do cheezein** dete hain: key aur default value (agar data na mile to wo value). Bhejne aur lene wali key bilkul **ek jaisi** honi chahiye.

4. Uri.parse()

URI ek **pata (address)** hai: website ka link, phone number, email. **Uri.parse()** text ko aisa pata bana deta hai jo Android samajh sake. Yeh sirf pata banata hai, khud kuch kholta nahi.

Definition (paper mein likho):
Uri.parse() converts a text string into a Uri object. It only parses the URI and does not open the resource by itself.

Syntax:

```
val uri = Uri.parse("text")
```

Example:

```
Uri.parse("https://www.example.com")   // website
Uri.parse("tel:03001234567")             // phone number
Uri.parse("mailto:user@example.com")    // email
```

5. Intent.createChooser()

Jab kayi apps ek kaam kar sakti hon (jaise share karna), to ek **list dikhata** hai 'kis app se share karna hai?'. Jaise aap ko pooch lena ke kis raaste se jao.

Definition (paper mein likho):
Intent.createChooser() creates an app-selection dialog that lets the user choose which application should handle an Intent.

Syntax:

```
Intent.createChooser(intent, "title")
```

Example:

```
val shareIntent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Welcome to Android")
}
val chooser = Intent.createChooser(shareIntent, "Choose an app")
startActivity(chooser)
```

10.5 Intent ke Actions (kaam batane wale)

Implicit Intent mein hum batate hain ke kaunsa kaam karna hai. 4 actions yaad rakho:

Action	Kaam	Data / URI
ACTION_VIEW	Website ya data dikhana (browser kholna).	Uri.parse("https://example.com")
ACTION_DIAL	Phone dialer kholna, number likha hua. User khud call karta hai (permission nahi chahiye).	Uri.parse("tel:03001234567")
ACTION_SEND	Text ya content kisi app ke saath share karna.	EXTRA_TEXT ke saath
ACTION_SENDTO	Email ya message app kholna (recipient ke saath).	Uri.parse("mailto:user@example.com")

```
// ACTION_DIAL
val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:03001234567"))
startActivity(intent)

// ACTION_SENDTO (email)
val email = Intent(Intent.ACTION_SENDTO, Uri.parse("mailto:student@example.com"))
email.putExtra(Intent.EXTRA_SUBJECT, "Android Assignment")
email.putExtra(Intent.EXTRA_TEXT, "Please find my assignment.")
startActivity(email)
```

Yaad rakho: Intent ke 2 types (Explicit, Implicit) aur 5 methods (startActivity, putExtra, getStringExtra, Uri.parse, createChooser) paper ke liye zaroor yaad karo.

Part 11: Mobile Development Trends (chhota section)

Yeh section chhota hai magar short question aa sakta hai. Har trend ko aik line + example se yaad karo:

Trend	Matlab	Example
AI-powered apps	Apps mein AI: chatbot, voice assistant, recommendations.	Shopping app jo aap ki pasand ki cheezein dikhaye.
Generative AI	AI jo text, image, code, summary khud banata hai.	AI tutor jo students ke sawal ka jawab de.
Cloud-based apps	Data aur services remote servers par (Firebase, AWS, Azure).	App -> Internet -> Cloud -> Database.
5G	Tez aur kam delay wala internet.	Real-time communication, cloud gaming, AR.
IoT	Apps ka doosre connected devices se rabta.	Smart home, smart watch, sensors.
AR (Augmented Reality)	Asal duniya mein digital cheezein shamil karna.	Furniture app: sofa apne kamray mein dekhna.
VR (Virtual Reality)	Poori nakli duniya mein dakhil hona.	Virtual training, tours, gaming.
Wearables	Smart watch, fitness band se rabta.	Fitness app smartwatch se data le.
Mobile security	Passwords, data, API keys ki hifazat: encryption, HTTPS, permissions, input validation.	Banking app.
Biometric authentication	Fingerprint ya face se login.	Fingerprint login.
Mobile payments	Digital wallets, QR, contactless.	Easypaisa, JazzCash.
On-device AI	AI phone ke andar chalna, server par bheje baghair.	Kam delay, behtar privacy, offline kaam.
Low-code / No-code	Kam ya bilkul code ke baghair app banana.	Tez prototype.

Trend	Matlab	Example
PWA / Cross-platform	Web jaisi installable app / ek code se Android + iOS.	Flutter, React Native.

AR vs VR: AR mein asal duniya mein digital cheezein **shamil** hoti hain. VR mein user **poori nakli duniya** mein chala jata hai.

Aakhir mein: Quick Revision aur Paper Tips

Topic	Yaad rakho
Kotlin	val (change nahi), var (change). println, readLine. if, when, for, while. fun, null safety (?), list, class, constructor, open, override, interface, data class.
Screen UI	XML = shakal, Kotlin = kaam. findViewById, setOnClickListener, .text.toString(), isChecked, tvResult.text = ...
Layouts	Linear (ek line, orientation), Relative (dusre ke relative, layout_below), Constraint (rassiyan, app:layout_constraint...).
Lifecycle (7)	onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy.
Architecture	Applications, Application Framework, ART + Native Libraries, HAL, Linux Kernel, Hardware.
4 Components	Activity, Service, Broadcast Receiver, Content Provider.
Android Studio	Official IDE. Kotlin = language, Studio = IDE, SDK = tools. WhatsApp/Facebook = native, Android + iOS, communication/social.
Intent	Explicit (naam se), Implicit (kaam se). startActivity, putExtra, getStringExtra, Uri.parse, createChooser.

Paper tips: (1) Har topic mein pehle **Definition**, phir **Syntax**, phir **Example** likho. (2) Code mein **import lines** aur **id** ke naam sahi rakho: XML ki id aur Kotlin mein R.id.xxx bilkul ek jaisi honi chahiye. (3) Screen wale sawal mein XML aur Kotlin **dono** likho, aur ek line mein batao app kaise chalti hai. (4) Architecture aur lifecycle ka **diagram** banao. (5) Layouts ka **comparison table** banao. Allah kare paper behtareen jaye, Moto!

Mids ke possible sawal: (1) Android Architecture ki layers diagram ke saath. (2) Activity lifecycle ke 7 methods, purpose aur usage. (3) LinearLayout, RelativeLayout, ConstraintLayout mein farq example ke saath. (4) Ek login/registration screen ka XML aur Kotlin code. (5) Student marks/grade/even-odd Kotlin program. (6) Explicit aur implicit intent, putExtra/getStringExtra. (7) WhatsApp aur Facebook kis category ki apps hain. (8) Android Studio kya hai. (9) val aur var mein farq. (10) CheckBox, RadioButton, Switch, Spinner kya hain.